


**ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA**

**GRADO EN INGENIERÍA INFORMÁTICA**

**Curso Académico 2025-2026**

**Trabajo de Fin de Grado**

# DETECCIÓN TEMPRANA DE FAKE NEWS EN TELEGRAM MEDIANTE ANÁLISIS AUTOMATIZADO Y METODOLOGÍA EXPERIMENTAL REPRODUCIBLE

**Autor:** Álvaro Osuna Flores

**Tutora:** Liliana Patricia Santacruz Valencia

# Tabla de contenido

Resumen ..... iv
1. Introducción ..... 1
1.1 Motivación ..... 1
1.2 Objetivos ..... 2
1.3 Estructura de la memoria ..... 3
2. Contexto ..... 4
2.1 Desinformación y clasificación automática ..... 4
2.2 Telegram como fuente de datos ..... 4
2.3 Datasets de referencia ..... 5
2.4 Representación textual y clasificadores lineales ..... 6
2.5 Evaluación, trazabilidad y ética de datos ..... 7
2.6 Trabajos relacionados y posicionamiento del proyecto ..... 8
3. Metodología, requisitos y arquitectura funcional ..... 12
3.1 Metodología de trabajo ..... 12
3.2 Requisitos funcionales ..... 12
3.3 Requisitos no funcionales ..... 13
3.4 Arquitectura funcional ..... 13
3.5 Herramientas y tecnologías ..... 14
3.6 Modelo de datos y gobierno del corpus ..... 14
3.7 Reproducibilidad y control de cambios ..... 15
3.8 Decisiones de diseño y justificación técnica ..... 16
4. Desarrollo e implementación ..... 20
4.1 Construcción del dataset unificado ..... 20
4.2 Validación del dataset ..... 21
4.3 Preprocesamiento textual ..... 22
4.4 Extracción de mensajes desde Telegram ..... 22
4.5 Configuración del entrenamiento ..... 23
4.6 Artefactos de trazabilidad ..... 24
4.7 Protocolo de reproducción ..... 25
4.8 Implementación por módulos ..... 25
4.9 Pruebas y control de calidad ..... 27
5. Evaluación y análisis de resultados ..... 28

5.1 Comparativa de modelos candidatos ..... 28
5.2 Métricas del modelo seleccionado ..... 28
5.3 Matriz de confusión ..... 29
5.4 Análisis cualitativo de errores ..... 30
5.5 Interpretabilidad del modelo lineal ..... 31
5.6 Validación automatizada ..... 31
5.7 Amenazas a la validez ..... 32
5.8 Lectura de resultados por escenario ..... 32
5.9 Criterios de aceptación del baseline ..... 33
6. Conclusiones y líneas futuras ..... 35
Bibliografía ..... 37
Glosario ..... 40
Apéndice A. Configuración del run de referencia ..... 41

## Resumen

Este Trabajo Fin de Grado desarrolla una base experimental reproducible para estudiar la detección temprana de fake news en Telegram desde una perspectiva propia de la Ingeniería Informática. El trabajo se concentra en la adquisición controlada de datos, la normalización textual, la construcción de un corpus verificable, el entrenamiento de clasificadores de referencia y el análisis trazable de sus errores.

La solución implementada integra extracción bajo demanda mediante Telethon, deduplicación persistente, limpieza textual determinista, validación automática de esquema y entrenamiento de dos modelos lineales sobre representaciones TF-IDF. El corpus final contiene 50.503 registros procedentes de archive2 y LIAR, con 23.470 textos etiquetados como verificado o neutro y 27.033 etiquetados como fake news. Los criterios de inclusión, exclusión, idioma, longitud textual y transformación se guardan en metadatos, de modo que el experimento pueda repetirse sin depender de decisiones.

El baseline seleccionado ha sido un Linear SVC entrenado sobre unigramas y bigramas, con selección por macro F1 y una política de umbral orientada a reducir falsos positivos. En el conjunto de validación estratificado de 12.626 ejemplos obtuvo accuracy = 0,9426, precisión macro = 0,9420, recall macro = 0,9439 y macro F1 = 0,9424. El análisis de la matriz de confusión, de los ejemplos de error y de los términos influyentes muestra que el modelo capta señales útiles de clasificación, aunque también aprende rasgos de estilo editorial y procedencia de fuente. Por ello, los resultados se interpretan como una base experimental sólida y no como una validación definitiva sobre mensajes reales de Telegram.

# 1. Introducción

La desinformación digital se ha consolidado como un problema relevante en el ecosistema informativo actual, debido a la facilidad con la que los contenidos falsos, manipulados o engañosos pueden producirse, difundirse y amplificarse en plataformas digitales. La literatura académica ha señalado que las fake news no solo afectan a la calidad de la información disponible, sino que también plantean retos técnicos, sociales y metodológicos para su detección automática (Lazer et al., 2018). Además, se ha observado que los contenidos falsos pueden alcanzar una difusión más rápida y amplia que los contenidos verificados, lo que incrementa la importancia de disponer de mecanismos de análisis temprano (Vosoughi et al., 2018).

Desde el punto de vista de la Ingeniería Informática, la detección automática de fake news no puede reducirse únicamente a entrenar un modelo de clasificación. Antes de interpretar cualquier resultado, es necesario controlar la procedencia de los datos, las transformaciones aplicadas al texto, la definición de las etiquetas, la calidad del corpus y el procedimiento de evaluación. Diversos trabajos han mostrado que la detección de fake news puede abordarse mediante señales de contenido, contexto social y propagación, pero también que cada enfoque exige un diseño experimental cuidadoso para evitar conclusiones poco reproducibles o sesgadas (Shu et al., 2017; Thorne & Vlachos, 2018).

Este Trabajo Fin de Grado se sitúa dentro de ese marco y propone una base experimental reproducible para estudiar la detección temprana de fake news en Telegram. La propuesta se centra en construir y validar un corpus textual, aplicar técnicas de preprocesamiento, entrenar clasificadores de referencia y conservar los artefactos necesarios para interpretar los resultados obtenidos. De esta forma, el proyecto no pretende presentar un sistema final de verificación automática, sino una metodología controlada que permita analizar el comportamiento de modelos supervisados sobre datos documentados, trazables y comparables con referencias existentes como LIAR (Wang, 2017).

## 1.1 Motivación

Telegram ofrece canales públicos, grupos masivos y una API accesible para desarrolladores. Esas características permiten analizar mensajes de forma automatizada, pero también facilitan

que contenidos de veracidad dudosa circulen con rapidez y con escasa mediación editorial. La literatura sobre desinformación ha mostrado que las noticias falsas pueden propagarse con mayor velocidad que las verificadas en redes sociales (Vosoughi et al., 2018), y que su detección automática requiere combinar análisis textual, conocimiento del dominio y evaluación rigurosa (Shu et al., 2017; Thorne y Vlachos, 2018).

Desde la Ingeniería Informática, el problema no se reduce a entrenar un clasificador. Antes de evaluar cualquier modelo, se necesita conocer de dónde proceden los datos, qué transformaciones se aplican, qué etiquetas se conservan, qué registros se descartan y cómo se reproducen los resultados. Si esas decisiones no quedan documentadas, una métrica elevada pierde valor académico, porque no puede distinguirse entre capacidad real del modelo, fuga de estilo, desequilibrio del corpus o error en la preparación de datos.

El presente TFG responde a esa necesidad con un pipeline experimental centrado en información textual. El sistema puede extraer mensajes desde Telegram, pero su aportación principal está en el control de datos y en la metodología de evaluación. Unifica fuentes externas, normaliza textos, valida el esquema, entrena baselines comparables, persiste artefactos por ejecución y analiza los errores con evidencias consultables. La solución se ha diseñado para que quien repita el experimento encuentre los mismos pasos, los mismos parámetros y los mismos puntos de control.

## 1.2 Objetivos

Los objetivos definidos para el presente proyecto se describen a continuación.

### Objetivo general

Desarrollar una base experimental reproducible para estudiar la detección temprana de fake news en Telegram, con foco en el procesamiento textual, la comparación de modelos lineales y la interpretación de resultados.

### Objetivos específicos

- Implementar una extracción bajo demanda de mensajes de canales públicos de Telegram, con reintentos, persistencia en JSON y reporte de canales fallidos.
- Diseñar un pipeline de preprocesamiento textual que normalice HTML, URLs, menciones, hashtags, caracteres no latinos, stopwords configurables y banderas de calidad.

- Construir un corpus binario unificado a partir de archive2 y LIAR, con criterios explícitos de inclusión, exclusión y mapeo de etiquetas.
- Validar automáticamente el dataset antes del entrenamiento, comprobando columnas obligatorias, nulos, duplicados, etiquetas válidas y coherencia entre etiqueta numérica y nombre de clase.
- Entrenar y comparar Regresión Logística y Linear SVC sobre representaciones TF-IDF, seleccionando el mejor modelo por macro F1.
- Persistir métricas, matriz de confusión, predicciones del holdout, ejemplos de aciertos y errores, términos influyentes y manifiestos de ejecución.
- Diferenciar el alcance del TFG de Informática respecto al TFG de Computadores, evitando duplicar el enfoque de despliegue, colas, API operacional y observabilidad distribuida.

## 1.3 Estructura de la memoria

La estructura de la memoria es como sigue:

El capítulo 2, Contexto, revisa el contexto teórico y técnico. El capítulo 3, Metodología, requisitos y arquitectura funcional, describe la metodología, los requisitos y la arquitectura funcional. El capítulo 4, Desarrollo e implementación, detalla el desarrollo, la construcción del corpus, el preprocesamiento, la extracción, el entrenamiento y la trazabilidad de artefactos. El capítulo 5, Evaluación y análisis de resultados, analiza los resultados, la matriz de confusión, la interpretabilidad y las limitaciones. El capítulo 6, Conclusiones y líneas futuras, recoge las conclusiones y las líneas de trabajo futuro.

# 2. Contexto

En este capítulo se describe el contexto teórico y tecnológico en el que se enmarca el presente Trabajo de Fin de Grado. El objetivo es proporcionar una base conceptual sólida para comprender la relevancia del problema abordado y los fundamentos de las decisiones tanto técnicas como morales de la elección del conjunto de datos utilizado.

## 2.1 Desinformación y clasificación automática

La desinformación se ha estudiado como un fenómeno comunicativo y técnico en el que intervienen intención, contenido, contexto de difusión y recepción social. Lazer et al. (2018) plantean que la ciencia de las fake news necesita métodos verificables, datos compartibles y análisis empírico. Desde una perspectiva computacional, Shu et al. (2017) agrupan los enfoques de detección en métodos basados en contenido, contexto social y propagación. Este trabajo se sitúa en la primera familia, ya que evalúa el texto como fuente principal de señal.

Los modelos basados en contenido textual suelen transformar documentos en vectores numéricos y después entrenar clasificadores supervisados. Las representaciones TF-IDF siguen siendo útiles cuando se necesita una línea base transparente, reproducible y de bajo coste computacional. Aunque los modelos de lenguaje preentrenados como BERT han mejorado muchas tareas de NLP (Devlin et al., 2019), un baseline lineal sigue siendo una referencia razonable para conocer el comportamiento del corpus antes de introducir modelos más complejos.

## 2.2 Telegram como fuente de datos

Telegram permite canales públicos con gran número de suscriptores, mensajes reenviados, publicaciones multimedia y acceso mediante API. Para un TFG centrado en datos, estas propiedades ofrecen una vía de adquisición controlada. No obstante, los mensajes reales de Telegram no tienen la misma distribución que los artículos periodísticos tradicionales ni que las declaraciones políticas breves de datasets de referencia. Esta diferencia de dominio condiciona la interpretación de los resultados y exige reconocer los límites del corpus utilizado.

La extracción implementada en este proyecto se ha planteado como herramienta auxiliar, no como sistema de vigilancia continua. Se ejecuta bajo demanda, respeta los canales configurados y genera ficheros JSON que pueden deduplicarse antes de entrar en el pipeline experimental.

## 2.3 Datasets de referencia

El corpus LIAR de Wang (2017) contiene declaraciones políticas etiquetadas en seis niveles de veracidad. Su utilidad reside en la granularidad de la anotación, aunque esa misma granularidad obliga a decidir cómo se mapea a un problema binario. El corpus archive2, por su parte, proporciona artículos completos etiquetados como reales o falsos. Al combinarlos, se obtiene un conjunto mayor y más variado, pero también aparecen riesgos de sesgo de fuente. Los modelos pueden aprender marcas editoriales en lugar de señales semánticas profundas (Gravanis et al., 2019).

La Tabla 1 sitúa este trabajo frente a soluciones y referencias relevantes. La comparación no pretende agotar el estado del arte, sino justificar por qué el proyecto se centra en trazabilidad experimental antes que en una arquitectura de producción.

**Tabla 1**

*Comparativa de referencias y soluciones relacionadas*

| Referencia | Enfoque | Datos | Aportación |
|---|---|---|---|
| Shu et al. (2017) | Revisión de minería de datos para fake news. | Redes sociales y contenido textual. | Marco conceptual para distinguir contenido, contexto y propagación. |
| Wang (2017) | Benchmark LIAR con etiquetas de veracidad. | Declaraciones políticas de Politifact. | Fuente externa para construir el corpus binario. |
| Ruchansky et al. (2017) | Modelo híbrido con contenido, respuesta social y fuente. | Twitter y Weibo. | Evidencia de que el texto aislado no captura todo el fenómeno. |
| Ozbay y Alatas (2020) | Clasificación supervisada de fake news. | Corpus de noticias online. | Referencia para comparar baselines clásicos. |
| TFG de Computadores | Pipeline operativo desacoplado. | Mensajes procesados y evidencias de sistema. | Referencia de acabado documental, no de alcance técnico para este TFG. |

*Nota.* La comparación se formula desde el alcance de Ingeniería Informática: datos, modelos y reproducibilidad.

## 2.4 Representación textual y clasificadores lineales

La representación TF-IDF se ha elegido porque permite construir un punto de partida medible y comprensible. En esta representación, cada documento se transforma en un vector donde los términos frecuentes en un texto reciben peso, pero ese peso disminuye si el término aparece en muchos documentos del corpus. La técnica no entiende el significado profundo de una frase, aunque sí captura regularidades léxicas que suelen ser suficientes para establecer un baseline en clasificación textual. Cuando se trabaja con datos heterogéneos, esa simplicidad ayuda a distinguir errores de modelado, errores de limpieza y problemas de distribución.

El uso de unigramas y bigramas amplía la información disponible para el clasificador. Un unigrama captura palabras aisladas, mientras que un bigrama conserva pares consecutivos. Esa decisión resulta relevante porque varias señales de procedencia o estilo aparecen como expresiones compuestas (washington reuters, read more, featured image u on wednesday). Si el vectorizador solo usara palabras aisladas, parte de esas señales quedaría fragmentada. Al incorporar bigramas, el modelo recibe patrones más cercanos a la forma real de los textos.

La Regresión Logística y el Linear SVC se han seleccionado por su adecuación a espacios de alta dimensión. Los vectores TF-IDF suelen tener miles de características, muchas de ellas dispersas. Los clasificadores lineales funcionan bien en ese contexto porque no necesitan construir fronteras no lineales complejas para separar documentos cuando existen señales léxicas informativas. Además, ambos modelos facilitan una interpretación global. Sus coeficientes permiten identificar términos asociados a cada clase, lo que resulta útil para detectar sesgos de fuente.

La Regresión Logística ofrece probabilidades que pueden compararse con un umbral configurable. Esa propiedad la hace cómoda cuando se pretende modular la sensibilidad del sistema. El Linear SVC, en cambio, optimiza el margen de separación entre clases y produce una función de decisión. Aunque esa puntuación no sea una probabilidad calibrada, se puede desplazar mediante un umbral para obtener una política de predicción más prudente. En este TFG, la comparación entre ambos enfoques permite seleccionar el modelo por rendimiento sin perder la trazabilidad de la decisión.

Conviene subrayar que el baseline lineal no se plantea como final del problema. Su función es establecer una referencia reproducible, ligera y explicable. Si un modelo transformer se incorporara después, debería superar esta línea base bajo las mismas particiones, las mismas

fuentes documentadas y un protocolo de evaluación comparable. De lo contrario, la mejora podría deberse a cambios en el dataset, en el umbral o en el procedimiento, no necesariamente al modelo.

## 2.5 Evaluación, trazabilidad y ética de datos

La evaluación de un sistema de clasificación binaria requiere distinguir entre aciertos, falsos positivos y falsos negativos. En un contexto de fake news, un falso positivo etiqueta como falso un contenido que el dataset considera verificado o neutro. Un falso negativo deja sin detectar un contenido etiquetado como fake_news. La gravedad relativa de ambos errores depende del uso previsto. Este TFG no automatiza decisiones sobre usuarios ni canales, pero sí documenta una política conservadora para evitar sobredetección en la clase positiva.

La trazabilidad se ha tratado como requisito metodológico. Cada ejecución guarda un manifiesto con identificador, hash del dataset, rutas de artefactos, modelo seleccionado, umbral y métricas. Ese manifiesto permite reconstruir qué se entrenó, con qué datos y bajo qué parámetros. Si dos ejecuciones obtienen métricas distintas, el investigador puede comparar hashes, configuración y artefactos antes de atribuir la diferencia al algoritmo. Este control es especialmente necesario cuando se trabaja con scripts que pueden regenerar datos y modelos.

El tratamiento de datos de Telegram se aborda desde una posición restrictiva. La extracción se limita a canales públicos configurados por el usuario del experimento, se ejecuta bajo demanda y no almacena más información de la necesaria para el análisis textual. Aunque el corpus principal procede de fuentes externas de referencia, el diseño evita normalizar la idea de una vigilancia continua. Si en una fase futura se anotaran mensajes reales de Telegram, deberían definirse criterios de anonimización, minimización, consentimiento cuando proceda y revisión ética del protocolo.

La documentación de limitaciones también forma parte de la ética de datos. Presentar un macro F1 elevado sin explicar sesgo de fuente, binarización de LIAR o ausencia de validación específica en Telegram induciría una lectura exagerada del alcance. Por ello, la memoria separa con cuidado rendimiento interno y capacidad de generalización. El resultado es útil para investigación y aprendizaje, pero todavía no autoriza un despliegue automático sobre comunidades reales.

## 2.6 Trabajos relacionados y posicionamiento del proyecto

La detección automática de fake news ha sido abordada desde distintas perspectivas dentro del procesamiento del lenguaje natural, la minería de datos y el análisis de redes sociales. Aunque todos estos enfoques comparten el objetivo general de identificar contenido potencialmente falso o engañoso, difieren en el tipo de información que utilizan, en el grado de complejidad del modelo y en las condiciones necesarias para su aplicación práctica. Por este motivo, resulta necesario situar el presente Trabajo de Fin de Grado dentro del conjunto de soluciones existentes y justificar el alcance técnico adoptado.

Una primera línea de trabajo se basa en el análisis textual del contenido. En este enfoque, el documento, noticia, mensaje o declaración se representa mediante características lingüísticas que posteriormente se introducen en un clasificador supervisado. Las representaciones clásicas, como bolsa de palabras o TF-IDF, permiten construir modelos sencillos, reproducibles y relativamente fáciles de interpretar. Esta aproximación ha sido utilizada de forma frecuente como línea base en tareas de clasificación textual, ya que ofrece un punto de comparación estable antes de introducir arquitecturas más complejas. Su principal ventaja es que permite analizar qué términos o expresiones influyen en la decisión del modelo, lo que facilita detectar posibles sesgos de fuente, marcas editoriales o patrones de estilo.

Dentro de esta línea, los clasificadores lineales como Regresión Logística y Linear SVC resultan especialmente adecuados cuando los textos se representan mediante vectores de alta dimensión. En problemas de clasificación de noticias falsas, estos modelos pueden alcanzar resultados competitivos cuando existen diferencias léxicas claras entre clases. Sin embargo, su rendimiento debe interpretarse con cautela, porque pueden aprender señales superficiales en lugar de razonar sobre la veracidad factual del contenido. Por ejemplo, un modelo puede asociar determinadas expresiones, cabeceras, firmas de agencia o fórmulas periodísticas a una clase concreta, aunque dichas señales no sean necesariamente indicativas de verdad o falsedad.

Una segunda línea de investigación incorpora modelos de lenguaje preentrenados, como BERT y otras arquitecturas basadas en transformers. Estos modelos representan un avance importante porque capturan dependencias contextuales más ricas que los enfoques basados únicamente en frecuencias de palabras. A diferencia de TF-IDF, un transformer puede generar representaciones distintas de una misma palabra en función de la frase en la que aparece. Esto permite abordar mejor fenómenos como la ambigüedad, la ironía o la composición semántica. No obstante, estos modelos también introducen mayores costes computacionales, menor

interpretabilidad directa y una dependencia más fuerte de las condiciones de entrenamiento y ajuste fino. Para un trabajo centrado en reproducibilidad y control experimental, resulta razonable establecer primero un baseline lineal antes de evaluar modelos más complejos.

Una tercera línea de trabajo no se limita al texto, sino que incorpora información social y de propagación. En plataformas digitales, una noticia falsa no se caracteriza únicamente por su contenido, sino también por la forma en la que se difunde: velocidad de propagación, número de reenvíos, estructura de la red, cuentas que amplifican el mensaje o comunidades en las que aparece. Estos enfoques pueden ser muy útiles para detectar campañas coordinadas o patrones anómalos de difusión. Sin embargo, requieren datos adicionales que no siempre están disponibles o que pueden plantear problemas de privacidad, acceso y ética. Además, en el caso de Telegram, la estructura de canales y grupos no ofrece siempre la misma visibilidad que otras redes sociales, lo que complica la reconstrucción completa de la propagación.

Una cuarta línea está relacionada con la detección multimodal. Muchas noticias falsas no se difunden solo como texto, sino acompañadas de imágenes, vídeos, capturas de pantalla, audios o enlaces externos. En estos casos, un sistema exclusivamente textual puede perder información relevante. Un mensaje aparentemente neutro puede remitir a una imagen manipulada, o una noticia falsa puede apoyarse en un vídeo sacado de contexto. Los enfoques multimodales combinan procesamiento de lenguaje natural, visión por computador y análisis de metadatos para obtener una representación más completa del contenido. A pesar de su interés, esta línea queda fuera del alcance del presente TFG, que se centra de forma deliberada en el componente textual y en la construcción de una metodología experimental controlada.

También existen trabajos orientados al fact-checking automático. En este tipo de sistemas, el objetivo no es solo clasificar un texto como falso o verdadero, sino contrastar afirmaciones concretas con fuentes externas, bases de conocimiento o documentos de referencia. Esta aproximación se acerca más al proceso humano de verificación, ya que requiere identificar afirmaciones verificables, recuperar evidencias y determinar si dichas evidencias apoyan, contradicen o no permiten decidir la afirmación analizada. Aunque esta línea es más ambiciosa desde el punto de vista semántico, también exige una infraestructura más compleja y criterios de evaluación diferentes. El presente trabajo no implementa fact-checking automático, sino clasificación supervisada sobre un corpus etiquetado.

El uso de datasets de referencia, como LIAR, permite comparar resultados con trabajos previos y disponer de etiquetas ya definidas. Sin embargo, estos conjuntos de datos no siempre

coinciden con el dominio final de aplicación. LIAR contiene declaraciones políticas breves, mientras que otros corpus pueden estar compuestos por artículos completos, titulares o noticias procedentes de medios concretos. Esta diferencia condiciona el aprendizaje del modelo. Un clasificador entrenado sobre un dataset puede obtener buenas métricas internas y, aun así, generalizar peor cuando se aplica a mensajes reales de Telegram. Por ello, una parte esencial del presente TFG consiste en documentar la procedencia de los datos, las reglas de transformación y las limitaciones del corpus utilizado.

El caso de Telegram introduce además particularidades propias. Los mensajes pueden ser breves, reenviados, fragmentados, acompañados de enlaces o redactados con un estilo menos formal que una noticia periodística tradicional. También pueden aparecer mensajes con errores ortográficos, abreviaturas, emojis, menciones, hashtags o referencias contextuales difíciles de interpretar fuera del canal en el que se publican. Estas características hacen que el traslado directo de modelos entrenados con datasets externos de noticias o declaraciones políticas deba considerarse una aproximación inicial, no una validación definitiva sobre el dominio objetivo.

Desde esta perspectiva, el presente proyecto se posiciona como una base experimental reproducible y no como un detector final de desinformación. Su objetivo principal no es maximizar una métrica aislada, sino construir un flujo controlado que permita adquirir datos, normalizarlos, validar el corpus, entrenar modelos de referencia, conservar artefactos y analizar los errores obtenidos. Esta decisión metodológica resulta relevante porque en problemas de fake news una métrica elevada puede ser engañosa si no se conoce la composición del corpus, el origen de los textos, el reparto de clases o las señales que el modelo está utilizando para clasificar.

La elección de TF-IDF, Regresión Logística y Linear SVC responde a este posicionamiento. Estos modelos permiten establecer una línea base clara, ligera y explicable. Además, facilitan la inspección de términos influyentes, lo que ayuda a detectar si el modelo está aprendiendo señales semánticas relacionadas con el contenido o marcas superficiales asociadas a determinadas fuentes. En este sentido, la interpretabilidad del baseline forma parte del propio diseño experimental. Antes de incorporar modelos más potentes, es necesario conocer qué patrones existen en el corpus y hasta qué punto las métricas obtenidas dependen de ellos.

El proyecto se diferencia, por tanto, de tres tipos de soluciones. En primer lugar, se diferencia de los sistemas operacionales de monitorización continua, porque la extracción desde Telegram se plantea bajo demanda y no como vigilancia persistente. En segundo lugar, se diferencia de

los sistemas avanzados de fact-checking, porque no recupera evidencias externas ni verifica afirmaciones individuales. En tercer lugar, se diferencia de los modelos profundos de clasificación, porque prioriza la reproducibilidad, la trazabilidad y la explicación de resultados sobre la complejidad algorítmica.

Esta delimitación no reduce el valor del trabajo, sino que lo concreta. En un problema sensible como la detección de desinformación, resulta preferible construir primero una base metodológica verificable que permita conocer las fortalezas y debilidades del pipeline. A partir de esa base, futuras ampliaciones podrían incorporar validación cruzada, particiones por fuente, mensajes reales de Telegram anotados manualmente, modelos transformer, análisis multimodal o estrategias de recuperación de evidencias. Sin embargo, todas esas mejoras deberían compararse con el baseline actual y mantener el mismo nivel de trazabilidad documental.

En consecuencia, el presente TFG se sitúa en el punto intermedio entre una revisión conceptual del problema y un sistema de producción. Su aportación consiste en transformar el problema de la detección temprana de fake news en un experimento reproducible, auditable y extensible. El sistema desarrollado no pretende decidir de forma automática la veracidad de los mensajes, sino proporcionar una infraestructura experimental desde la que estudiar el comportamiento de modelos supervisados y establecer una referencia sólida para trabajos posteriores.

# 3. Metodología, requisitos y arquitectura funcional

## 3.1 Metodología de trabajo

El proyecto se ha desarrollado de forma incremental, con fases verificables y artefactos persistidos. Primero se preparó la adquisición de mensajes y la deduplicación. Después se construyó el corpus unificado y se añadieron validaciones automáticas. En una tercera fase se entrenaron modelos candidatos y persistieron sus resultados. Finalmente se analizó el comportamiento del baseline mediante métricas, matriz de confusión, ejemplos de error y pesos del modelo lineal.

Cada fase puede repetirse de manera aislada. Esta independencia importa porque permite corregir el dataset sin reescribir el entrenamiento, modificar la política de umbral sin cambiar el preprocesamiento o evaluar un modelo persistido sin reconstruir todo el corpus. Si se quisiera ampliar el trabajo con otro dataset, bastaría con mantener el esquema y volver a ejecutar la validación antes del entrenamiento.

## 3.2 Requisitos funcionales

La Tabla 2 recoge los requisitos funcionales que guían la implementación.

**Tabla 2**

*Requisitos funcionales del sistema experimental*

| Código | Descripción |
|---|---|
| RF-1 | Extraer mensajes de canales públicos de Telegram y mantenerlos en formato JSON cuando existan credenciales válidas. |
| RF-2 | Deduplicar mensajes mediante identificadores de canal y mensaje para evitar repeticiones entre sesiones. |
| RF-3 | Normalizar texto, tokenizar, detectar idioma y marcar banderas de calidad por registro. |
| RF-4 | Construir un corpus binario unificado desde archive2 y LIAR con criterios documentados. |
| RF-5 | Validar esquema, etiquetas, nulos y duplicados antes de iniciar el entrenamiento. |
| RF-6 | Entrenar Regresión Logística y Linear SVC sobre TF-IDF y seleccionar el modelo con mayor macro F1. |
| RF-7 | Persistir por ejecución los artefactos de evaluación y análisis necesarios para reproducir resultados. |

*Nota.* RF = Requisito Funcional.

## 3.3 Requisitos no funcionales

La Tabla 3 resume los requisitos no funcionales, que se centran en reproducibilidad, trazabilidad, legibilidad y separación de alcance.

**Tabla 3**

*Requisitos no funcionales del sistema experimental*

| Código | Descripción |
|---|---|
| RNF-1 | La ejecución debe ser reproducible a partir de ficheros de configuración versionados. |
| RNF-2 | Cada run debe tener un identificador único que agrupe métricas, predicciones, matriz de confusión y manifiesto. |
| RNF-3 | La validación del dataset debe detener el flujo si detecta incoherencias críticas. |
| RNF-4 | El diseño debe mantener bajo acoplamiento entre adquisición, preprocesamiento, entrenamiento y evaluación. |
| RNF-5 | El sistema debe evitar dependencias operacionales propias del TFG de Computadores, como cola, API de consulta y dashboards. |
| RNF-6 | Las salidas deben ser interpretables y aptas para análisis académico, no solo para ejecución automatizada. |

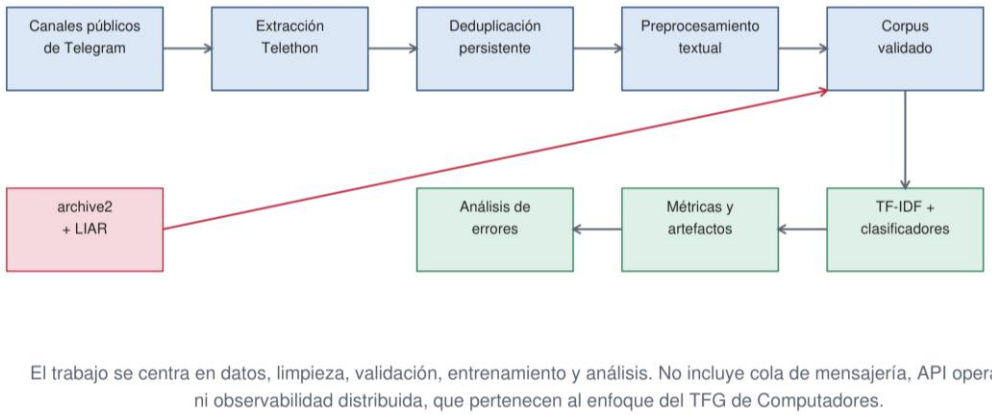
*Nota.* RNF = Requisito No Funcional.

## 3.4 Arquitectura funcional

La arquitectura funcional se organiza como un flujo de datos reproducible. La Figura 1 muestra cómo se conectan la extracción opcional desde Telegram, la deduplicación, el preprocesamiento, el corpus validado, el entrenamiento y los artefactos de análisis. La entrada de archive2 y LIAR se incorpora en la construcción del corpus, no en un flujo operativo de tiempo real.

**Figura 1**

*Arquitectura funcional centrada en datos y experimentación*




*Nota.* Elaboración propia a partir de los módulos del proyecto y de los artefactos versionados del run baseline-20260411T145313Z-a9ea359a.

## 3.5 Herramientas y tecnologías

La Tabla 4 resume las tecnologías empleadas y su función dentro de este TFG.

**Tabla 4**

*Herramientas y tecnologías utilizadas*

| Categoría | Herramienta | Función |
|---|---|---|
| Adquisición | Telethon | Acceso programático a canales públicos de Telegram. |
| Preprocesamiento | re, html, langdetect | Limpieza, tokenización, normalización y detección de idioma. |
| Datos | pandas, csv, zipfile | Construcción del corpus y lectura de fuentes externas. |
| Modelado | scikit-learn | TF-IDF, Regresión Logística, Linear SVC y métricas. |
| Persistencia | JSON, CSV, joblib | Manifiestos, predicciones, modelos y resultados. |
| Pruebas | pytest | Validación automatizada de módulos y flujos principales. |
| Documentación | Word/PDF, APA7 | Memoria final con tablas, figuras, citas y referencias. |

*Nota.* TF-IDF = Term Frequency-Inverse Document Frequency.

## 3.6 Modelo de datos y gobierno del corpus

El modelo de datos del corpus se ha diseñado para conservar tanto información original como información derivada. La columna text mantiene el contenido de entrada, mientras que normalized_text almacena la versión procesada que alimenta el vectorizador. Esta separación evita que la limpieza borre evidencia útil para análisis posteriores. Si un error de clasificación

depende de una URL sustituida, de una mención o de un hashtag normalizado, el investigador puede volver al texto original y comprobar qué información se perdió o se transformó.

La columna source_id actúa como identificador estable de cada registro. En fuentes externas, el identificador combina origen y posición. En extracción desde Telegram, se puede derivar del canal y del identificador del mensaje. Esta decisión permite detectar duplicados, enlazar predicciones del holdout con ejemplos concretos y revisar errores sin depender del orden de las filas. Si se regenerase el dataset, source_id debería seguir siendo estable para facilitar comparaciones entre ejecuciones.

Las columnas label y label_name se guardan de forma redundante. Aunque podría bastar con la etiqueta numérica, mantener el nombre de clase reduce errores de interpretación durante el análisis manual. La validación comprueba que ambos campos sean coherentes: 0 corresponde a verificado_o_neutro y 1 corresponde a fake_news. Este control sencillo evita un fallo frecuente en experimentación, que consiste en invertir etiquetas sin advertirlo hasta que las métricas ya se han calculado.

El campo dataset_name permite analizar la procedencia de cada ejemplo. En este trabajo resulta especialmente importante, porque los errores y los términos influyentes muestran diferencias claras entre archive2 y LIAR. Si el modelo funciona bien por reconocer marcas de una fuente concreta, dataset_name permite detectarlo. En una ampliación futura, este campo debería usarse para particiones por fuente, de modo que el rendimiento se midiera en datos que no compartan estilo editorial con el entrenamiento.

Los metadatos del corpus cumplen una función de gobierno documental. No se limitan a describir tamaños; registran archivos usados, archivos descartados, reglas de limpieza, etiquetas excluidas, distribución de idiomas, resumen de longitudes y hash del dataset. Esa información permite reconstruir por qué el corpus tiene su forma actual. También impone una disciplina: cuando se cambie una regla de construcción, deberá actualizarse el metadato y se generará un nuevo hash.

## 3.7 Reproducibilidad y control de cambios

La reproducibilidad se ha implementado en tres niveles. El primer nivel es la configuración, donde los parámetros de entrenamiento viven en un fichero JSON, no dispersos en instrucciones manuales. El segundo nivel es el versionado de artefactos, donde cada run crea una carpeta propia y no sobrescribe resultados anteriores. El tercer nivel es la validación automática, donde

el pipeline no confía en que el dataset esté bien formado, sino que lo comprueba antes de entrenar.

El uso de random_state = 42 en la partición holdout fija el reparto de datos. Esta decisión no convierte el resultado en una estimación universal, pero sí permite repetir exactamente el experimento de referencia. Para una evaluación más robusta sería deseable añadir validación cruzada o particiones por fuente, pero el run fijo tiene valor pedagógico y documental. Establece una base común contra la que se puede medir cualquier modificación posterior.

El hash SHA-256 del dataset es una pieza central del control de cambios. Dos ficheros con el mismo nombre pueden contener datos distintos, especialmente cuando se regeneran a partir de fuentes externas. Al guardar el hash en el manifiesto, el documento de resultados deja de depender solo de la ruta del fichero. Si el hash cambia, el investigador sabe que las métricas ya no corresponden al mismo corpus y debe generar una nueva interpretación.

La memoria se ha redactado con esa misma lógica. Las tablas y figuras citan la procedencia de los datos (metadata, comparison, classification_report, confusion_matrix, prediction_examples, linear_model_terms y training_manifest). De esta forma, el lector puede rastrear cada cifra hasta un artefacto concreto. Este criterio reduce ambigüedad y evita que la memoria se convierta en una narración desvinculada del código ejecutado.

## 3.8 Decisiones de diseño y justificación técnica

El diseño del proyecto se ha basado en una serie de decisiones técnicas orientadas a mantener el equilibrio entre funcionalidad, reproducibilidad, interpretabilidad y alcance académico. En un problema como la detección de fake news, existen múltiples alternativas posibles: modelos clásicos, modelos profundos, análisis de propagación, fact-checking automático, extracción continua de canales o integración con sistemas operacionales. Sin embargo, no todas esas alternativas resultan adecuadas para el objetivo de este Trabajo de Fin de Grado. Por ello, esta sección recoge las principales decisiones adoptadas y justifica por qué se han considerado apropiadas para el alcance del proyecto.

La primera decisión relevante ha sido centrar el trabajo en una base experimental reproducible y no en un sistema de producción. Esta elección condiciona toda la arquitectura. En lugar de

desplegar una aplicación permanente que monitorice canales, procese mensajes en tiempo real y emita alertas automáticas, el proyecto se organiza como un pipeline ejecutable bajo demanda. Esta aproximación permite controlar mejor los datos utilizados, conservar los artefactos generados y repetir el experimento en condiciones equivalentes. Desde el punto de vista académico, esta trazabilidad resulta más importante que disponer de una interfaz final, ya que permite justificar cómo se han obtenido las métricas y qué limitaciones tienen.

La segunda decisión ha sido separar claramente las fases de adquisición, construcción del corpus, validación, entrenamiento y evaluación. Esta separación reduce el acoplamiento entre módulos y facilita la detección de errores. Si la extracción de Telegram falla, no afecta directamente al entrenamiento sobre el corpus ya construido. Si se modifica una regla de preprocesamiento, puede regenerarse el dataset y producir un nuevo hash. Si se cambia la configuración del modelo, se crea un nuevo run sin sobrescribir los resultados anteriores. Esta organización modular permite que cada fase tenga una responsabilidad concreta y que los resultados puedan auditarse con mayor claridad.

La tercera decisión ha sido utilizar fuentes externas de referencia para construir el corpus principal. Aunque el título del trabajo se centra en Telegram, no se ha asumido que los mensajes extraídos desde canales públicos estén automáticamente etiquetados. Para entrenar modelos supervisados es necesario disponer de etiquetas de clase, y obtenerlas directamente de Telegram exigiría un protocolo de anotación manual, criterios de consenso y revisión ética adicional. Por este motivo, se ha optado por construir un corpus binario a partir de datasets externos, documentando sus reglas de inclusión y exclusión. La extracción de Telegram queda así planteada como una capacidad auxiliar y extensible, no como la fuente principal de verdad del entrenamiento.

La cuarta decisión ha sido formular el problema como una clasificación binaria. Esta simplificación permite distinguir entre contenidos etiquetados como fake_news y contenidos considerados verificados o neutros. Aunque algunos datasets, como LIAR, contienen etiquetas de veracidad más granulares, utilizar todas las clases habría aumentado la complejidad del problema y dificultado la interpretación de resultados. La binarización permite construir una primera línea base clara, pero también introduce una limitación metodológica: las categorías intermedias de veracidad no siempre encajan de forma natural en una clase positiva o negativa. Por ello, las clases ambiguas se han excluido en lugar de forzar su asignación.

La quinta decisión ha sido conservar tanto el texto original como el texto normalizado. El campo original permite revisar posteriormente qué información contenía cada registro antes del preprocesamiento, mientras que el campo normalizado permite alimentar el vectorizador de forma homogénea. Esta doble representación evita perder trazabilidad. Si un ejemplo se clasifica de forma incorrecta, puede analizarse si el error procede del contenido original, de la limpieza aplicada, de la longitud del texto, de la presencia de URLs, menciones o hashtags, o de señales propias de la fuente. En consecuencia, el preprocesamiento no se plantea como una operación destructiva, sino como una transformación documentada.

La sexta decisión ha sido utilizar TF-IDF como técnica principal de representación textual. Aunque existen representaciones más avanzadas, TF-IDF ofrece varias ventajas para este proyecto. En primer lugar, es una técnica ligera y suficientemente estable para datasets de tamaño medio. En segundo lugar, permite entrenar modelos lineales con bajo coste computacional. En tercer lugar, facilita la interpretación de los pesos asociados a los términos. Esta última característica es especialmente importante, ya que permite comprobar si el modelo aprende señales relacionadas con el contenido o si se apoya en marcas editoriales, fórmulas de estilo o patrones propios de una fuente concreta.

La séptima decisión ha sido comparar dos modelos lineales: Regresión Logística y Linear SVC. Ambos son adecuados para espacios vectoriales de alta dimensión y datos dispersos, como los generados por TF-IDF. La Regresión Logística proporciona una salida probabilística que facilita aplicar umbrales sobre la clase positiva. Linear SVC, por su parte, optimiza una frontera de separación basada en margen y suele comportarse bien en tareas de clasificación textual. Comparar ambos modelos bajo la misma partición de datos permite seleccionar el baseline de forma objetiva, evitando elegir un único algoritmo sin contraste experimental.

La octava decisión ha sido seleccionar el modelo mediante macro F1. Esta métrica calcula el rendimiento medio entre clases sin ponderar únicamente por el tamaño de cada una. En un problema de fake news, esta decisión resulta adecuada porque no interesa que el modelo funcione bien solo en la clase mayoritaria. Accuracy puede ser útil como referencia global, pero puede ocultar desequilibrios entre precisión y recall. Macro F1 permite observar si el clasificador mantiene un comportamiento razonablemente equilibrado entre contenidos verificados o neutros y contenidos etiquetados como fake_news.

La novena decisión ha sido incorporar una política de umbral conservadora. En este contexto, un falso positivo implica clasificar como fake_news un contenido que el dataset considera

verificado o neutro. Aunque el proyecto no despliega un sistema real ni toma decisiones sobre usuarios, este tipo de error tiene una implicación relevante: puede sobredetectar contenido legítimo. Por ello, se ha preferido una configuración prudente que reduzca la clasificación positiva dudosa, aunque eso implique aceptar algunos falsos negativos adicionales. Esta decisión no es universal, sino dependiente del escenario de uso. En un sistema de alerta temprana para revisión humana podría ajustarse de otro modo.

La décima decisión ha sido persistir todos los artefactos relevantes de cada ejecución. El proyecto no se limita a mostrar métricas por pantalla, sino que guarda manifiestos, predicciones del holdout, matriz de confusión, comparativa de modelos, términos influyentes y ejemplos de predicción. Esta persistencia convierte cada entrenamiento en un experimento auditable. Si en el futuro se modifica el dataset, el preprocesamiento o el modelo, será posible comparar los nuevos resultados con el run de referencia. Además, conservar estos ficheros facilita que las cifras incluidas en la memoria puedan rastrearse hasta una fuente concreta.

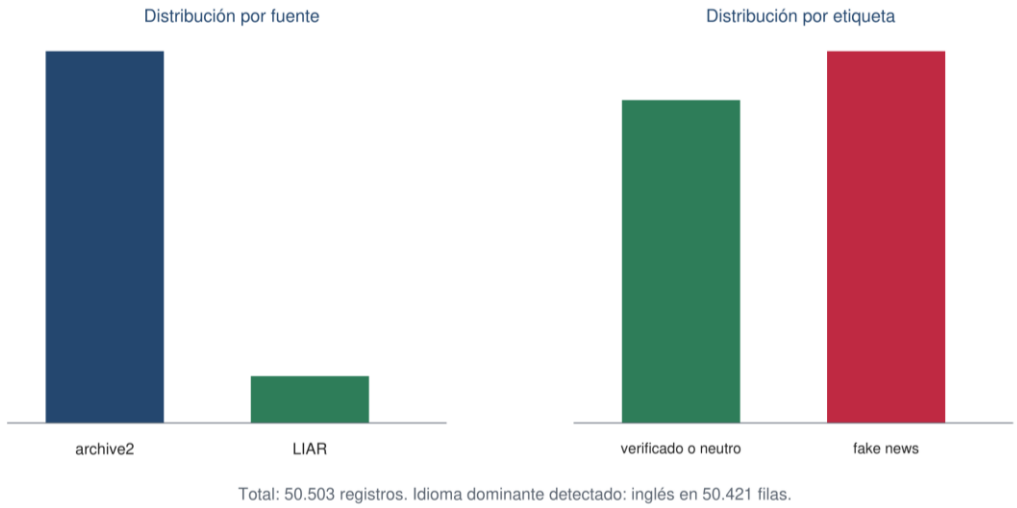
# 4. Desarrollo e implementación

## 4.1 Construcción del dataset unificado

La construcción del dataset se realiza mediante *scripts/build_real_dataset.py*. El proceso lee las fuentes externas, aplica reglas de mapeo de etiquetas, descarta registros incompatibles y genera data/labeled/fake_news_unified.csv.gz. La Figura 2 representa la composición final del corpus y permite observar que archive2 domina el conjunto, mientras que LIAR aporta declaraciones políticas más breves.

**Figura 2**

*Composición del corpus por fuente y etiqueta*




*Nota.* Datos procedentes de fake_news_unified.metadata.json, generado el 9 de abril de 2026.

La Tabla 5 detalla las reglas de inclusión y exclusión. Esta información resulta necesaria porque las métricas del modelo dependen de la forma en que se transforma un problema multiclase, como LIAR, en una tarea binaria.

**Tabla 5**

*Criterios de inclusión y exclusión aplicados al corpus*

| Elemento | Decisión | Justificación |
|---|---|---|
| archive2.zip | Incluido. | Aporta artículos completos con etiquetas Fake y True. |
| archive.zip (LIAR) | Incluido parcialmente. | Se usan solo true, false y pants-fire para mantener una tarea binaria clara. |
| archive3.zip | Excluido. | Presenta incompatibilidad de esquema con las fuentes incorporadas. |

| | | |
|---|---|---|
| LIAR half-true | Excluido. | Se descartan 2.627 filas por ambigüedad intermedia. |
| LIAR mostly-true | Excluido. | Se descartan 2.454 filas por no encajar de forma limpia en la clase positiva o negativa. |
| LIAR barely-true | Excluido. | Se descartan 2.103 filas por representar un grado parcial de veracidad. |
| Filas vacías | Excluidas. | Se descartan 2 filas sin texto útil. |

*Nota.* Las reglas proceden de fake_news_unified.metadata.json.

La Tabla 6 resume las características cuantitativas del dataset final.

**Tabla 6**

*Características del dataset unificado*

| Característica | Valor |
|---|---|
| Total de filas | 50.503 |
| Clase verificado_o_neutro | 23.470 (46,5%) |
| Clase fake_news | 27.033 (53,5%) |
| Filas de archive2 | 44.898 (88,9%) |
| Filas de LIAR | 5.605 (11,1%) |
| Idioma dominante | en (50.421 filas) |
| Media de caracteres crudos | 2278,58 |
| Media de tokens normalizados | 374,76 |
| SHA-256 del dataset | cfaf1b39c0260008e9328950649c9b83... |

*Nota.* Los porcentajes se calculan sobre 50.503 filas.

## 4.2 Validación del dataset

La validación se ejecuta antes del entrenamiento. `scripts/validate_dataset.py` comprueba que existan las columnas obligatorias, que no haya valores nulos en campos críticos, que source_id no esté duplicado, que las etiquetas pertenezcan al conjunto permitido y que label_name sea coherente con label. Si alguna condición falla, el entrenamiento no se inicia. Esta puerta de calidad reduce el riesgo de que un resultado aparentemente bueno provenga de un error de preparación.

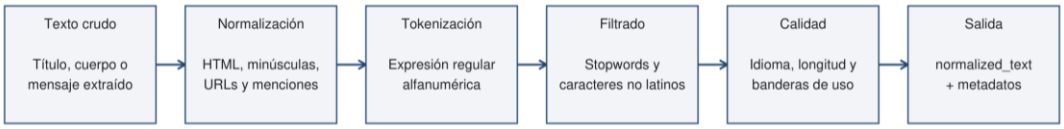
La validación no debe interpretarse como garantía de verdad semántica de las etiquetas. Lo que garantiza es consistencia estructural: el dataset tiene el formato esperado y puede alimentar el experimento. La calidad epistemológica de las etiquetas procede de las fuentes originales y de las decisiones de mapeo documentadas.

## 4.3 Preprocesamiento textual

El módulo *preprocessing.py* transforma cada texto crudo en una representación normalizada. La Figura 3 muestra las etapas del pipeline. La normalización conserva trazabilidad, ya que no sustituye los campos originales por los derivados. El texto crudo, la fuente, el identificador y la etiqueta permanecen disponibles para auditoría.

**Figura 3**

*Etapas del pipeline de preprocesamiento textual*




La limpieza no borra la trazabilidad. El registro conserva la fuente, el identificador, la etiqueta original y los campos derivados necesarios para reproducir el entrenamiento.

*Nota.* Elaboración propia a partir de las reglas persistidas en fake_news_unified.metadata.json.

La Tabla 7 describe las reglas de limpieza aplicadas.

**Tabla 7**

*Reglas de preprocesamiento documentadas*

| Regla | Efecto |
|---|---|
| R1 | Convierte entidades HTML y normaliza el texto a minúsculas. |
| R2 | Sustituye URLs por el token 'url' y menciones por el token 'usuario'. |
| R3 | Conserva el termino base de los hashtags y elimina el simbolo '#'. |
| R4 | Elimina emojis y caracteres fuera de la clase latina configurada. |
| R5 | Compacta espacios en blanco y tokeniza con una expresión regular alfanumerica. |
| R6 | Filtra stopwords configurables por entorno antes de reconstruir normalized_text. |
| R7 | Marca textos vacíos, demasiado cortos o con idioma desconocido mediante quality_flags. |

*Nota.* Las reglas quedan persistidas junto al corpus para que la transformación pueda revisarse sin abrir el código fuente.

## 4.4 Extracción de mensajes desde Telegram

*telegram_extractor.py* implementa la adquisición desde canales públicos mediante Telethon. El módulo usa credenciales configuradas por entorno, gestiona reintentos y produce

un informe de canales fallidos. La extracción se ha diseñado para ejecutarse cuando el investigador lo decida, de manera que el corpus pueda enriquecerse con ejemplos de Telegram sin convertir este TFG en un sistema de captura permanente.

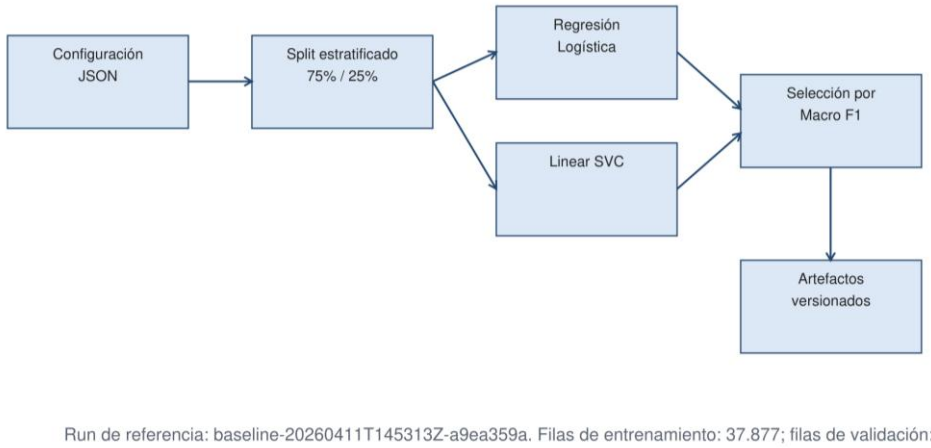
*dataset_manager.py* añade deduplicación persistente. Cada mensaje queda identificado por canal e identificador interno, lo que evita que sesiones sucesivas introduzcan duplicados. Esta decisión es relevante porque el entrenamiento supervisado se ve afectado por repeticiones. Un mensaje duplicado puede aparecer en entrenamiento y validación, aumentar artificialmente las métricas y distorsionar el análisis de errores.

## 4.5 Configuración del entrenamiento

El entrenamiento se configura en configs/training_config.json. La Figura 4 sintetiza el flujo. El dataset validado se divide en entrenamiento y validación, se entrenan los dos candidatos, se aplica la política de umbral y se selecciona el modelo por macro F1. La elección de macro F1 evita favorecer automáticamente la clase mayoritaria y permite comparar el rendimiento medio en ambas clases.

**Figura 4**

*Flujo de entrenamiento y evaluación reproducible*




*Nota.* Elaboración propia a partir de training_config.json y training_manifest.json.

La configuración usa una partición holdout estratificada con test_size = 0.25 y random_state = 42. La vectorización TF-IDF emplea unigramas y bigramas, min_df = 1 y max_features = 5000. Los modelos candidatos son Regresión Logística y Linear SVC, ambos con class_weight = balanced. La política de predicción establece probability_threshold = 0.6 para modelos probabilísticos y decision_threshold = 0.3 para modelos basados en función de decisión.

## 4.6 Artefactos de trazabilidad

Cada ejecución genera un directorio propio bajo reports/training_runs. La Figura 5 muestra una captura estructurada del manifiesto de entrenamiento. Este manifiesto enlaza configuración, dataset, hash, modelo seleccionado, rutas de artefactos y métricas agregadas. Así, si se modifica el corpus o el umbral, el cambio queda asociado a un nuevo run_id.

**Figura 5**

*Captura del manifiesto de entrenamiento reproducible*

training_manifest.json - extracto del run reproducible


```
{
    "run_id": "baseline-20260411T145313Z-a9ea359a",
    "dataset_sha256": "cfaf1b39c0260008e9328950...",
    "text_column": "normalized_text",
    "label_column": "label",
    "best_model": "linear_svc",
    "selection_metric": "macro_f1",
    "train_rows": 37877,
    "holdout_rows": 12626,
      "probability_threshold": 0.6,
      "decision_threshold": 0.3
}
```


La captura muestra los campos mínimos que conectan configuración, dataset, modelo y evaluación.

*Nota.* Captura generada a partir del fichero training_manifest.json del run baseline-20260411T145313Z-a9ea359a.

La Tabla 8 enumera los artefactos generados y su utilidad.

**Tabla 8**

*Artefactos persistidos por ejecución*

| Artefacto | Contenido | Uso en la memoria |
|---|---|---|
| training_manifest.json | Configuración, rutas, hash del dataset y métricas agregadas. | Trazabilidad del experimento. |
| comparison.json | Resultados de todos los modelos candidatos. | Comparativa de baselines. |
| classification_report.json | Precisión, recall y F1 por clase. | Análisis cuantitativo desglosado. |
| confusion_matrix.json | Matriz de confusión estructurada. | Interpretación de falsos positivos y falsos negativos. |

| | | |
|---|---|---|
| holdout_predictions.csv | Predicción de cada ejemplo del conjunto de validación. | Revisión granular y auditoría. |
| prediction_examples.json | Ejemplos representativos de aciertos y errores. | Análisis cualitativo. |
| linear_model_terms.json | Términos con mayor peso por clase. | Interpretabilidad y detección de sesgo de fuente. |

## 4.7 Protocolo de reproducción

El protocolo de reproducción debe ejecutarse en orden, porque cada paso consume artefactos del anterior. La Tabla 9 resume el flujo mínimo. Si se cambia el dataset o la configuración, conviene conservar el run anterior y generar uno nuevo, ya que comparar resultados sobre condiciones distintas exige mantener ambos registros.

**Tabla 9**

*Protocolo mínimo para reproducir el experimento*

| Paso | Comando | Resultado esperado |
|---|---|---|
| 1 | python -m scripts.build_real_dataset | Genera fake_news_unified.csv.gz y sus metadatos. |
| 2 | python -m scripts.validate_dataset --input data/labeled/fake_news_unified.csv.gz | Verifica esquema, etiquetas y duplicados. |
| 3 | python -m scripts.train_baseline | Entrena candidatos y crea reports/training_runs/<run_id>. |
| 4 | python -m scripts.evaluate_baseline --manifest reports/training_runs/<run_id>/training_manifest.json | Regenera evaluación de un modelo persistido. |
| 5 | python -m pytest -q | Comprueba los módulos principales mediante pruebas automatizadas. |

## 4.8 Implementación por módulos

La implementación se ha dividido en módulos pequeños, cada uno con una responsabilidad clara. Esta organización facilita probar partes concretas del sistema y evita que la lógica de entrenamiento se mezcle con la adquisición de datos. Aunque el proyecto no persigue una arquitectura de microservicios, sí aplica separación funcional: extraer, normalizar, construir, validar, entrenar, evaluar y analizar son actividades distintas.

*telegram_extractor.py* encapsula la comunicación con Telegram mediante Telethon. El módulo no decide etiquetas ni aplica entrenamiento; su salida es una colección de mensajes con metadatos mínimos. Esta separación tiene una consecuencia, si en el futuro cambiara la fuente de adquisición, el resto del pipeline podría mantenerse siempre que se entregue un fichero compatible. El extractor, por tanto, actúa como adaptador de entrada.

*dataset_manager.py* gestiona la deduplicación persistente. La deduplicación no se ha implementado como un filtrado puntual en memoria, porque las sesiones de extracción pueden repetirse en días distintos. Mantener un registro de identificadores ya vistos evita que el mismo mensaje se incorpore varias veces al conjunto crudo. Además, permite que la extracción sea incremental sin degradar la calidad del dataset.

*preprocessing.py* contiene la lógica de limpieza textual. El módulo devuelve un objeto con texto normalizado, tokens, recuento de tokens, idioma detectado, banderas de calidad e indicador de usabilidad. Esta salida estructurada resulta preferible a devolver solo una cadena, porque permite conservar información diagnóstica. Un texto demasiado corto o con idioma desconocido no se confunde con un texto normalizado válido; queda marcado para que las fases siguientes decidan cómo tratarlo.

*scripts/build_real_dataset.py* integra las fuentes externas. Su responsabilidad no es entrenar, sino construir el corpus canónico y escribir metadatos. Esta separación evita que el entrenamiento tenga que conocer detalles internos de archive2 o LIAR. Si se añadiera una tercera fuente, el cambio debería concentrarse en la construcción del dataset y en la documentación de reglas, no en el clasificador.

*scripts/validate_dataset.py* actúa como contrato entre datos y modelo. El entrenamiento no debería aceptar un fichero solo porque exista en la ruta esperada. Debe verificar columnas, duplicados, nulos y etiquetas. Esta filosofía convierte errores de datos en fallos tempranos y comprensibles, en lugar de permitir que aparezcan como métricas anómalas o excepciones tardías dentro de scikit-learn.

*scripts/train_baseline.py* coordina la parte experimental. Lee configuración, carga el dataset, crea la partición holdout, entrena candidatos, calcula métricas, selecciona el mejor modelo y persiste artefactos. El script no imprime únicamente resultados finales; deja ficheros que pueden revisarse después. Esta persistencia es la diferencia entre una prueba exploratoria y un experimento documentable.

*analysis_utils.py* agrupa funciones de análisis complementario. Su papel es generar matriz de confusión, ejemplos representativos y términos influyentes. Mantener estas tareas fuera del script principal reduce complejidad y permite probarlas por separado. También facilita que la memoria use artefactos ya generados en lugar de recalcular manualmente estadísticas durante la redacción.

*scripts/evaluate_baseline.py* permite evaluar un modelo persistido a partir de su manifiesto. Esta decisión es útil cuando se quiere comprobar que el modelo guardado y los artefactos siguen siendo coherentes. También permite distinguir dos escenarios: entrenamiento de un nuevo baseline y evaluación de un baseline ya existente. En trabajos académicos, esa distinción evita confundir ajuste de modelo con verificación posterior.

## 4.9 Pruebas y control de calidad

La estrategia de pruebas se ha orientado a cubrir los puntos donde un fallo tendría mayor impacto sobre los resultados. El preprocesamiento se prueba con entradas vacías, URLs, menciones, hashtags, textos cortos y casos de idioma desconocido. Estas pruebas reducen el riesgo de que una regla aparentemente simple, como reemplazar URLs, destruya contenido útil o genere tokens inesperados.

Las pruebas de construcción del dataset verifican que el mapeo de etiquetas y los descartes se apliquen de forma consistente. En una tarea de fake news, un error en el mapeo true, false o pants-fire cambiaría el significado completo del experimento. Por eso, esas reglas no deben quedar solo en la memoria, deben estar codificadas y cubiertas por tests.

Las pruebas de entrenamiento comprueban que un dataset sintético mínimo produzca un manifiesto y artefactos esperados. Aunque ese dataset no mida rendimiento real, sí valida la integración entre configuración, entrenamiento, métricas y persistencia. Si una refactorización rompe la generación de comparison.json o confusion_matrix.json, el fallo se detecta antes de redactar nuevas conclusiones.

El control de calidad documental se apoya en la coherencia entre artefactos y texto. Cada cifra incorporada a la memoria procede de un JSON o CSV concreto. Este criterio ha guiado la ampliación del documento. Las figuras de distribución, métricas, matriz de confusión, términos influyentes y manifiesto se han generado desde los artefactos del proyecto. La memoria no introduce capturas decorativas, sino evidencias vinculadas a lo que se explica.

# 5. Evaluación y análisis de resultados

## 5.1 Comparativa de modelos candidatos

La comparativa se realizó sobre 12.626 ejemplos de validación. La Figura 6 visualiza las métricas de los dos modelos candidatos. Las diferencias son pequeñas, pero Linear SVC supera a la Regresión Logística en accuracy, precisión macro, recall macro y macro F1.

**Figura 6**

*Comparativa de métricas entre Regresión Logística y Linear SVC*

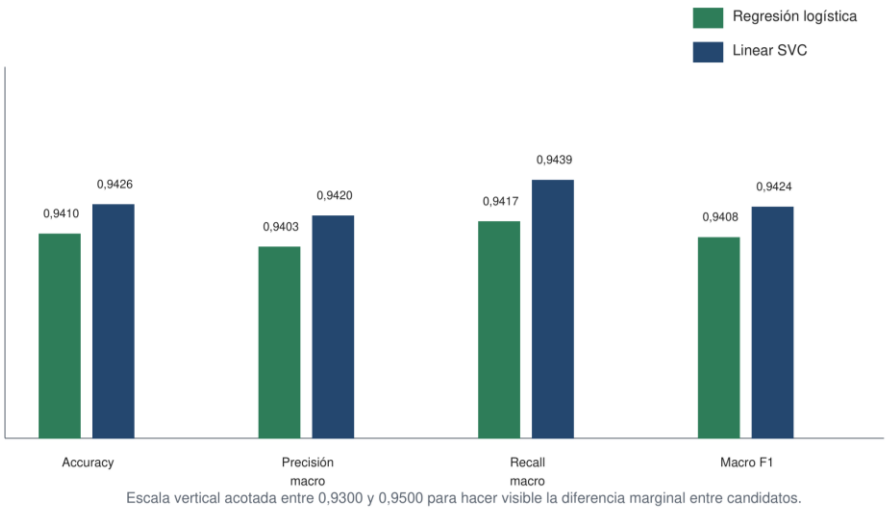



**Tabla 10**

*Comparativa de modelos candidatos*

| Modelo | Accuracy | Precisión macro | Recall macro | Macro F1 | Umbral usado |
|---|---|---|---|---|---|
| logistic_regression | 0,9410 | 0,9403 | 0,9417 | 0,9408 | 0,60 |
| linear_svc | 0,9426 | 0,9420 | 0,9439 | 0,9424 | 0,30 |

## 5.2 Métricas del modelo seleccionado

El Linear SVC obtiene un comportamiento equilibrado en ambas clases. La Tabla 11 recoge las métricas globales y por clase. La precisión de fake_news es superior al recall, lo que refleja la

política de umbral conservadora: el modelo marca menos positivos dudosos, aunque deja algunos casos falsos sin detectar.

**Tabla 11**

*Métricas del Linear SVC en el conjunto de validación*

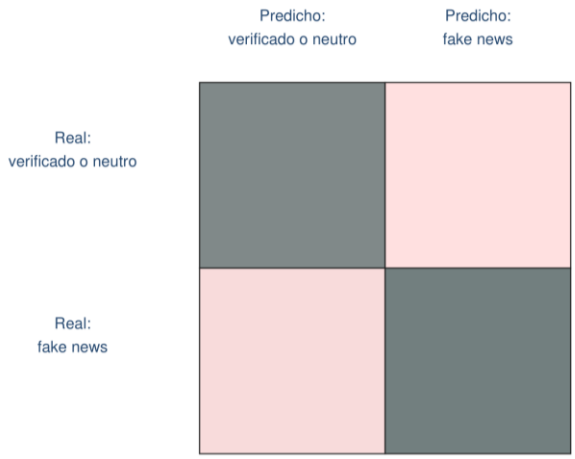
| Métrica | Global | verificado_o_neutro | fake_news |
|---|---|---|---|
| Accuracy | 0,9426 | - | - |
| Precisión | 0,9420 | 0,9179 | 0,9660 |
| Recall | 0,9439 | 0,9625 | 0,9253 |
| F1 | 0,9424 | 0,9397 | 0,9452 |
| Soporte | 12.626 | 5.868 | 6.758 |

## 5.3 Matriz de confusión

La Figura 7 y la Tabla 12 muestran la matriz de confusión. El modelo clasifica correctamente 5.648 ejemplos verificados o neutros y 6.253 ejemplos de fake news. Los errores se distribuyen en 220 falsos positivos y 505 falsos negativos.

**Figura 7**

*Matriz de confusión del Linear SVC*




La política de umbral deja 220 falsos positivos y 505 falsos negativos sobre 12.626 ejemplos de validación.

**Tabla 12**

*Matriz de confusión del modelo seleccionado*

| Real \ Predicho | verificado_o_neutro | fake_news |
|---|---|---|
| verificado_o_neutro | 5.648 | 220 |
| fake_news | 505 | 6.253 |

La lectura de esta matriz debe vincularse con la política de umbral. Un umbral más bajo aumentaría la sensibilidad hacia fake_news, pero también incrementaría el número de falsos positivos. En este TFG se ha preferido una configuración que reduzca sobredetección, porque el objetivo consiste en construir una base experimental interpretable, no en maximizar alarmas sobre un flujo operativo.

## 5.4 Análisis cualitativo de errores

prediction_examples.json permite revisar ejemplos concretos de cada cuadrante de la matriz. La Tabla 13 sintetiza los patrones observados. No se reproducen textos largos completos para evitar sobrecargar la memoria; se describen rasgos lingüísticos y de procedencia que explican el comportamiento del baseline.

**Tabla 13**

*Patrones detectados en los ejemplos de predicción*

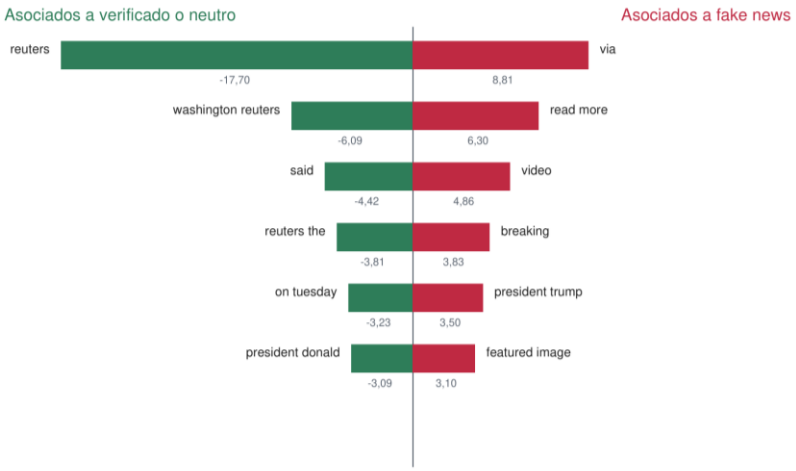
| Tipo | Patrón observado | Interpretación |
|---|---|---|
| Verdaderos positivos | Titulares con marcas virales, referencias a video, lectura ampliada y lenguaje polarizado. | El modelo reconoce señales frecuentes en artículos etiquetados como fake_news. |
| Verdaderos negativos | Textos con estilo de agencia, presencia de Reuters, verbos declarativos y fechas. | El modelo asocia marcas editoriales formales a verificado_o_neutro. |
| Falsos positivos | Declaraciones políticas breves de LIAR con léxico fuerte, aunque mapeadas a verificado_o_neutro. | El contenido breve y asertivo se parece estadísticamente a ejemplos de fake_news. |
| Falsos negativos | Textos falsos redactados con apariencia factual o con referencias de agencia. | El estilo periodístico reduce la puntuación positiva del clasificador. |

## 5.5 Interpretabilidad del modelo lineal

La Figura 8 muestra los términos con mayor peso absoluto en el Linear SVC. Los términos asociados a la clase verificado_o_neutro incluyen reuters, washington reuters y said. En cambio, fake_news queda asociada a expresiones como via, read more, video, breaking o featured image. Este resultado confirma que el modelo aprovecha señales de formato y procedencia, no solo contenido semántico.

**Figura 8**

*Términos con mayor peso por clase en el Linear SVC*




La interpretabilidad global del modelo lineal aporta una ventaja metodológica, permite detectar sesgos de fuente que podrían quedar ocultos si solo se informara la métrica macro F1. En este caso, la presencia de Reuters y de fórmulas de agencia revela que parte del rendimiento se apoya en marcas editoriales. Esta observación no invalida el baseline, pero obliga a usarlo como referencia experimental y no como detector final desplegable.

## 5.6 Validación automatizada

La suite de pruebas con pytest cubre preprocesamiento, deduplicación, extracción, configuración, construcción del dataset, entrenamiento, evaluación y análisis de artefactos. Las pruebas no reemplazan la evaluación estadística del modelo, pero protegen la integridad del pipeline. Si una modificación rompe la generación del manifiesto, cambia la política de umbral o elimina artefactos esperados, el fallo aparece antes de interpretar métricas.

El test de entrenamiento sobre un dataset sintético comprueba que se crea un manifiesto, que la configuración se lee correctamente, que se generan artefactos de análisis y que las rutas quedan asociadas al run. Esta cobertura resulta suficiente para el alcance del TFG porque los componentes son scripts reproducibles, no servicios persistentes.

## 5.7 Amenazas a la validez

La Tabla 14 recoge las principales amenazas a la validez. Identificarlas permite interpretar los resultados con prudencia y proponer trabajo futuro coherente.

**Tabla 14**

*Amenazas a la validez del estudio*

| Amenaza | Descripción | Mitigación aplicada o propuesta |
|---|---|---|
| Validez interna | El modelo puede aprender marcas de fuente en lugar de señales de veracidad. | Análisis de términos influyentes y reconocimiento explícito del source bias. |
| Validez externa | El corpus no procede de un conjunto anotado específicamente en Telegram. | Plantear un corpus futuro de Telegram con protocolo de anotación. |
| Validez de constructo | La binarización de LIAR simplifica etiquetas intermedias. | Excluir clases ambiguas y documentar los conteos descartados. |
| Validez estadística | Una única partición holdout puede depender del random_state. | Usar validación cruzada o particiones por fuente en extensiones futuras. |
| Validez de implementación | Errores en limpieza o validación podrían alterar el entrenamiento. | Pruebas automatizadas y metadatos de limpieza persistidos. |

## 5.8 Lectura de resultados por escenario

Los resultados pueden leerse de forma distinta según el escenario de uso. En un escenario puramente académico, el macro F1 obtenido confirma que el pipeline completo funciona y que el corpus contiene señales discriminativas suficientes para entrenar un baseline. En este escenario, lo más valioso no es solo la cifra final, sino la capacidad de repetir el proceso y examinar por qué se producen los aciertos y los errores.

En un escenario de prefiltrado para revisión humana, la política de umbral tendría que acordarse con los revisores. Si se quisiera evitar que contenidos falsos pasaran sin marcar, se bajaría el umbral y aumentaría el recall de fake_news. Si se quisiera reducir el volumen de alertas injustificadas, se mantendría o elevaría el umbral. El run de referencia opta por una posición conservadora frente a falsos positivos, pero no presenta esa elección como universal.

En un escenario de despliegue automático, el resultado todavía sería insuficiente. Faltan validación con mensajes reales de Telegram, análisis temporal, control por fuente, calibración, revisión ética y medición de impacto sobre usuarios. Además, la presencia de sesgo de fuente exige un protocolo que impida al modelo apoyarse en marcas editoriales ausentes en Telegram. Esta cautela no debilita el trabajo, lo sitúa dentro de su alcance real.

En un escenario de investigación futura, la memoria aporta una base de comparación. Un nuevo modelo debería reportar sus métricas con el mismo corpus o explicar claramente las diferencias. Un nuevo dataset debería conservar metadatos de construcción comparables. Una nueva política de umbral debería mostrar cómo cambia la matriz de confusión. Ese uso incremental es el principal valor del baseline.

## 5.9 Criterios de aceptación del baseline

El baseline puede considerarse aceptable para el alcance del TFG porque cumple cinco condiciones. En primer lugar, se entrena sobre un corpus documentado y validado. En segundo lugar, compara al menos dos modelos candidatos bajo la misma configuración de datos. En tercer lugar, selecciona por una métrica apropiada para clases no idénticas en tamaño. En cuarto lugar, persiste artefactos suficientes para auditar el resultado. En quinto lugar, reconoce limitaciones metodológicas en lugar de extrapolar indebidamente.

No obstante, aceptar el baseline no equivale a cerrarlo. El resultado debería revisarse si el corpus cambiara, si se añadieran mensajes reales de Telegram, si se modificara el preprocesamiento o si se ajustara el umbral. Cada una de esas modificaciones afecta al significado de las métricas. Por ello, la aceptación se vincula al run baseline-20260411T145313Z-a9ea359a y al hash de dataset incluido en el manifiesto.

También debe rechazarse cualquier lectura que presente el modelo como detector general de desinformación. El modelo distingue patrones aprendidos en archive2 y LIAR bajo una configuración concreta. Esa distinción puede ser útil como primera aproximación, pero no resuelve tareas de verificación factual, análisis multimodal, detección de campañas coordinadas

ni clasificación contextual de canales. Esas tareas pertenecen a fases posteriores y requerirían datos y métodos adicionales.

La conclusión operativa es sencilla: use este baseline como referencia reproducible, no como sustituto de una evaluación en el dominio objetivo. Si en una ampliación el modelo se aplica a Telegram real, primero deberá construirse un conjunto anotado, definirse un protocolo de revisión y medir la transferencia de dominio. Manténgase el baseline actual como control; así se podrá saber si las mejoras posteriores responden a mejor modelado o simplemente a cambios en los datos.

# 6. Conclusiones y líneas futuras

A partir del desarrollo realizado, puede afirmarse que los objetivos planteados al inicio del trabajo se han alcanzado de forma satisfactoria. En primer lugar, se ha construido una base experimental reproducible para el estudio de la detección temprana de fake news en Telegram, centrada en el tratamiento de datos textuales, la validación del corpus y la evaluación de modelos supervisados.

Además, se ha implementado un flujo completo que permite extraer mensajes de canales públicos, aplicar procesos de deduplicación, normalizar el texto y conservar los artefactos necesarios para revisar cada ejecución. También se ha logrado construir un corpus binario unificado a partir de fuentes externas, documentando los criterios de inclusión, exclusión y transformación de etiquetas. Asimismo, se han entrenado y comparado modelos lineales basados en TF-IDF, seleccionando como baseline final el modelo Linear SVC por su mejor rendimiento en macro F1.

Finalmente, se han generado métricas, matriz de confusión, ejemplos de predicción, términos influyentes y manifiestos de ejecución, lo que permite interpretar los resultados de manera trazable. Por tanto, el trabajo cumple el objetivo principal de proporcionar una metodología controlada, verificable y reutilizable para analizar la clasificación automática de desinformación textual, sin presentar el sistema como una solución definitiva de verificación, sino como una base sólida para futuras ampliaciones.

El trabajo ha construido una base experimental reproducible para estudiar la detección temprana de fake news en Telegram. La aportación principal no reside en desplegar una plataforma de producción, sino en ordenar un flujo completo de datos: adquisición controlada, deduplicación, normalización, construcción de corpus, validación, entrenamiento, evaluación e interpretación. Ese foco corresponde al perfil de Ingeniería Informática y se mantiene separado del TFG de Computadores.

El corpus final de 50.503 registros proporciona una base suficiente para entrenar un baseline lineal. Linear SVC ha sido el modelo seleccionado, con macro F1 = 0,9424 sobre el holdout estratificado. La matriz de confusión muestra un comportamiento sólido, aunque condicionado por la política de umbral: se reducen falsos positivos y aumentan falsos negativos. Esta decisión

es coherente con una primera criba experimental, donde se busca evitar una sobredetección difícil de justificar.

El análisis de términos influyentes revela una limitación importante. El modelo aprende señales como reuters, said, read more, video o featured image. Algunas proceden de estilos editoriales concretos y no de propiedades universales de la desinformación. En consecuencia, el resultado no debe presentarse como una solución final para Telegram real. Debe leerse como una base reproducible sobre la que se podrán diseñar experimentos más exigentes.

Las contribuciones del TFG son: un dataset binario documentado, reglas de limpieza persistidas, validación previa al entrenamiento, comparación de baselines lineales, selección por macro F1, análisis de matriz de confusión, revisión cualitativa de errores, interpretación de pesos y protocolo de reproducción. Si se quisiera continuar el proyecto, convendría preservar esta base como referencia mínima antes de incorporar modelos más costosos.

Las líneas futuras más relevantes son las siguientes:

- Construir un corpus anotado específicamente con mensajes de Telegram, definiendo criterios de inclusión, anonimización, doble anotación y resolución de desacuerdos.
- Separar entrenamiento y validación por fuente o por periodo temporal para medir con mayor rigor el efecto del sesgo editorial.
- Añadir análisis de errores por tema, longitud, fuente y presencia de URLs, de modo que el comportamiento del modelo pueda interpretarse con mayor granularidad.
- Evaluar calibración de puntuaciones y curvas de umbral antes de elegir políticas de decisión para contextos de revisión humana.
- Explorar modelos de lenguaje como DistilBERT o RoBERTa solo después de fijar el baseline TF-IDF, para que la mejora pueda medirse contra una referencia estable.
- Incorporar explicabilidad local mediante LIME o SHAP y contrastarla con la interpretabilidad global de pesos lineales.

## Bibliografía

Conroy, N. J., Rubin, V. L., y Chen, Y. (2015). Automatic deception detection: Methods for finding fake news. Proceedings of the Association for Information Science and Technology, 52(1), 1-4.

Devlin, J., Chang, M. W., Lee, K., y Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. En J. Burstein et al. (Eds.), Proceedings of NAACL-HLT 2019 (pp. 4171-4186). Association for Computational Linguistics.

Europol. (2022). Internet Organised Crime Threat Assessment (IOCTA) 2022. European Union Agency for Law Enforcement Cooperation.

Gravanis, G., Vakali, A., Diamantaras, K., y Karadais, P. (2019). Behind the cues: A benchmarking study for fake news detection. Expert Systems with Applications, 128, 201-213.

Guo, Y., Wang, D., Wang, L., Fang, Y., Wang, C., Yang, M., Liu, T., y Wang, H. (2024). Beyond app markets: Demystifying underground mobile app distribution via Telegram. Proceedings of the ACM on Measurement and Analysis of Computing Systems, 8(3), 1-25.

Jin, Z., Cao, J., Guo, H., Zhang, Y., y Luo, J. (2017). Multimodal fusion with recurrent neural networks for rumor detection on microblogs. Proceedings of the 25th ACM International Conference on Multimedia (pp. 795-816). ACM.

Kula, S., Choraś, M., Kozik, R., Ksieniewicz, P., y Woźniak, M. (2020). Application of the BERT-based architecture in fake news detection. En Proceedings of the International Conference on Artificial Intelligence in Medicine (pp. 138-145). Springer.

Lazer, D. M. J., Baum, M. A., Benkler, Y., Berinsky, A. J., Greenhill, K. M., Menczer, F., y Zittrain, J. L. (2018). The science of fake news. Science, 359(6380), 1094-1096.

Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., y Stoyanov, V. (2019). RoBERTa: A robustly optimized BERT pretraining approach. arXiv preprint arXiv:1907.11692.

Martini, F., Samula, P., Pickle, T., y Tucker, J. (2022). Bot detection in Telegram. Social Media + Society, 8(2), 1-13.

Nguyen, V. H., Sugiyama, K., Nakov, P., y Kan, M. Y. (2020). FANG: Leveraging social context for fake news detection using graph representation. Proceedings of the 29th ACM International Conference on Information and Knowledge Management (pp. 1165-1174). ACM.

Ozbay, F. A., y Alatas, B. (2020). Fake news detection within online social media using supervised artificial intelligence algorithms. Physica A: Statistical Mechanics and its Applications, 540, 123174.

Patwa, P., Sharma, S., Pykl, S., Guptha, V., Kumari, G., Akhtar, M. S., y Chakraborty, T. (2021). Fighting an infodemic: COVID-19 fake news dataset. En Combating Online Hostile Posts in Regional Languages during Emergency Situation (pp. 21-29). Springer.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., y Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

Ruchansky, N., Seo, S., y Liu, Y. (2017). CSI: A hybrid deep model for fake news detection. Proceedings of the 2017 ACM on Conference on Information and Knowledge Management (pp. 797-806). ACM.

Shu, K., Sliva, A., Wang, S., Tang, J., y Liu, H. (2017). Fake news detection on social media: A data mining perspective. ACM SIGKDD Explorations Newsletter, 19(1), 22-36.

Telegram. (2024). Telegram for business. https://telegram.org

Thorne, J., y Vlachos, A. (2018). Automated fact checking: Task formulations, methods and future directions. Proceedings of the 27th International Conference on Computational Linguistics (pp. 3346-3359). ACL.

UNESCO. (2018). Journalism, fake news and disinformation: Handbook for journalism education and training. United Nations Educational, Scientific and Cultural Organization.

Vosoughi, S., Roy, D., y Aral, S. (2018). The spread of true and false news online. Science, 359(6380), 1146-1151.

Wang, W. Y. (2017). "Liar, liar pants on fire": A new benchmark dataset for fake news detection. Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers, pp. 422-426). ACL.

Lazer, D. M. J., Baum, M. A., Benkler, Y., Berinsky, A. J., Greenhill, K. M., Menczer, F., Metzger, M. J., Nyhan, B., Pennycook, G., Rothschild, D., Schudson, M., Sloman, S. A., Sunstein, C. R., Thorson, E. A., Watts, D. J., & Zittrain, J. L. (2018). The science of fake news. *Science, 359*(6380), 1094–1096. https://doi.org/10.1126/science.aao2998

Shu, K., Sliva, A., Wang, S., Tang, J., & Liu, H. (2017). Fake news detection on social media: A data mining perspective. *ACM SIGKDD Explorations Newsletter, 19*(1), 22–36. https://doi.org/10.1145/3137597.3137600

Thorne, J., & Vlachos, A. (2018). Automated fact checking: Task formulations, methods and future directions. En *Proceedings of the 27th International Conference on Computational Linguistics* (pp. 3346–3359). Association for Computational Linguistics. https://aclanthology.org/C18-1283/

Vosoughi, S., Roy, D., & Aral, S. (2018). The spread of true and false news online. *Science, 359*(6380), 1146–1151. https://doi.org/10.1126/science.aap9559

Wang, W. Y. (2017). "Liar, liar pants on fire": A new benchmark dataset for fake news detection. En *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)* (pp. 422–426). Association for Computational Linguistics. https://doi.org/10.18653/v1/P17-2067

# Glosario

**Accuracy.** Proporción de ejemplos correctamente clasificados sobre el total evaluado.

**Baseline.** Modelo de referencia sencillo que permite comparar futuras mejoras con una base estable.

**Corpus.** Conjunto organizado de textos utilizado para entrenamiento o evaluación.

**Fake news.** Contenido falso, engañoso o manipulado que se presenta con apariencia informativa.

**Holdout.** Conjunto de validación separado del entrenamiento para estimar rendimiento.

**LIAR.** Dataset de declaraciones políticas etiquetadas por niveles de veracidad.

**Linear SVC.** Clasificador lineal de vectores de soporte, adecuado para datos textuales de alta dimensión.

**Macro F1.** Promedio no ponderado del F1 calculado por clase.

**N-grama.** Secuencia de una o varias palabras consecutivas utilizada como característica textual.

**Source bias.** Sesgo producido cuando un modelo aprende marcas de procedencia de los datos en lugar de señales propias de la tarea.

**TF-IDF.** Esquema de ponderación que combina frecuencia del término e inversa de frecuencia documental.

**Umbral de decisión.** Valor que transforma una puntuación continua del modelo en una etiqueta discreta.

# Apéndice A. Configuración del run de referencia

El run de referencia utilizado en esta memoria es baseline-20260411T145313Z-a9ea359a. Fue generado el 2026-04-11T14:55:05.542736+00:00 y usa el hash de dataset cfaf1b39c0260008e9328950649c9b83de2a9eadb478f631fb9cbc68d1b0a6b1. La Tabla A.1 resume los parámetros más relevantes.

**Tabla A.1**

*Parámetros principales del run de referencia*

| Parámetro | Valor |
|---|---|
| Dataset | \tfg_informatica_fake_news\data\labeled\fake_news_unified.csv.gz |
| Columna de texto | normalized_text |
| Columna de etiqueta | Label |
| Métrica de selección | macro_f1 |
| Mejor modelo | linear_svc |
| Tipo de puntuación | decision_function |
| Umbral aplicado | 0,30 |
| Filas de entrenamiento | 37.877 |
| Filas de validación | 12.626 |

*Nota.* Los valores proceden de training_manifest.json.